TTB Label Verifier

Approach and design

Matthew Nolan · June 2026 · live demo · github.com/Mjnolan91/ttb-label-verifier

1. Summary

Here is the whole tool in one paragraph. A TTB agent uploads photos of an alcohol label. An AI model reads the photos and writes down what the label says: the brand name, the alcohol content, the net contents, the government warning, and the rest of the fields TTB cares about. Then ordinary code, not the AI, compares each field against what the applicant claimed and against TTB's rules, and shows a verdict of Approve, Needs review, or Reject, with a short explanation under every field. A second screen does the same thing for a few hundred labels at once, and everything downloads as JSON or CSV.

The fastest way to judge it is to use it. Open the live demo and click "Load the sample label". A real front-and-back whisky label loads, the AI reads both photos together, and accepting its suggestions walks you to an Approve. Then click "Load the defective-warning version". The same flow ends in a Reject, and the reason names the regulation: the warning prefix on that label is printed in title case, and 27 CFR 16.22(a)(2) requires capitals and bold. One of the interviewed agents described rejecting labels for exactly that mistake; the demo replays her scenario live.

Three numbers tell you most of the story. The test suite is 826 tests, and it runs offline in a few seconds with no API keys, so anyone can verify the behavior on a laptop. The evaluation gate in CI fails the build if precision on approvals drops below 98 percent across 27 labeled cases; it scores 100 today. I set that gate because the two kinds of mistake are not equal: sending a fine label to review wastes minutes, while quietly approving a bad one would sink the tool's credibility. And speed: the deployed demo answers in a median of 3.1 seconds against the brief's rough five-second limit; the slowest of fifteen measured reads took 5.2, and that was the first one, while the server was still waking up. The bundled sample runs longer: six to nine seconds clean, because it is a front-and-back pair with twice the label to read, and about thirteen with the defect, because the tool takes one extra zoomed look at the warning before committing to a hard Reject.

2. What the customers said they wanted

The brief included notes from four discovery interviews, and each person gave me something specific to build against.

Sarah, the Deputy Director, cared most about speed. Her office had already tried a scanning vendor whose tool took 30 to 40 seconds per label, and her agents simply stopped using it, so anything slower than about five seconds is dead on arrival. She also told me who would use it: agents ranging from fresh graduates to a 73-year-old she considers her benchmark user. And she told me about the workload: importers drop 200 to 300 applications on the office at once.

Marcus, from IT, explained why the last vendor really died: the outbound firewall blocks third-party machine-learning endpoints, and they are an Azure shop. He also set the boundaries for a prototype: standalone, no personal data stored, and no integration with COLA, TTB's label-approval filing system.

Dave, an agent for 28 years, warned me about false rejections. STONE'S THROW and Stone's Throw are obviously the same brand to a person, and a tool that compares strings blindly would flood his queue with bogus rejections.

Jenny, a junior agent, described the strictest check she performs. The government warning must match the statute word for word, the "GOVERNMENT WARNING:" prefix must be in capitals and bold, and a title-case prefix gets the label rejected. She also asked that badly photographed labels fail gracefully instead of producing nonsense.

Most of those became features. Jenny's last request became a principle that shaped everything else: the tool may admit it is not sure, but it may never be quietly wrong.

3. How I implemented it

The AI reads, the code decides. The model's only job is transcription: look at the photos and write down what the label says. Everything that decides comes after, in ordinary unit-tested TypeScript. The brand comparison forgives case and punctuation, so STONE'S THROW equals Stone's Throw, and a near miss goes to a human instead of an automatic reject. A rules table knows which elements each beverage class must carry. The legally exact text, like the government warning and the per-class alcohol tolerances, lives in one hand-checked module that the tests guard. I built it this way because a rejection has to be explainable to an auditor after the fact: the reason is always a rule with a citation.

Confidence is measured, not asked for. A model will happily misread a blurry number and report full confidence while doing it, so I do not use self-reported confidence at all. Each product is read several times in parallel instead, and a field's confidence is simply how many of those reads agree. The front and back photos travel in one request, so the model sees the whole product at once. When a field stays doubtful after the vote, the tool escalates in small bounded steps: a stronger model re-reads just the doubtful fields; a dedicated check judges whether the warning prefix is truly bold; and if the warning seems missing, the tool crops that region of the photo, straightens it, enlarges it, and looks again. An escalation can clear a false alarm. It can never overturn a conflict and quietly pass the label.

The screens stay out of the agent's way. The single-label screen leads with the comparison the brief asked about: label versus application. The AI's reading appears as gray suggestions inside the form, and the agent accepts each with Tab or all of them with one click, so nobody retypes what the label already says. Nothing counts until a person accepts it, so the model never approves its own work. The batch screen runs the same engine over a whole folder: photos pair up by filename, an optional CSV supplies what each applicant claimed, and results stream into a worklist the agent can triage. If a label read cleanly but a required field came back empty, the tool quietly retries just those fields about twelve seconds later and flags anything it finds for review.

Everything runs offline by default. The default AI provider is a mock that recognizes the bundled test images, which means the 826 tests in 64 files and the 27-label evaluation run in seconds, deterministically, with no keys and no network. CI runs the identical gate on every push to main: types, lint, tests, evaluation, build. Clone the repo, run npm test, and the whole pipeline proves itself on your machine before you ever add an API key.

4. Why I chose this architecture

The model split: a fast reader and an expensive judge. My first instinct was to run everything on the strongest model available, so I tried that and measured it. The verdicts did not change at all, but the median time more than doubled, from 2.8 seconds to 6.5. When I repeated the

experiment on Google's models, the strong model also started failing requests outright (4 of 9) on its per-minute quota at my key's tier. Against Sarah's five-second limit, that settled it. Reading printed text is the part today's vision models are already good at, so the affordable model (gpt-4.1) does all of the reading, and the expensive one (gpt-5.5) is saved for the few judgments where one call can hard-fail a label, like whether the warning prefix is bold. The same pattern held on both vendors I measured, so the slowdown seems to come from the problem itself rather than from any one vendor's setup.

A provider interface, because lock-in already killed the last tool. The previous vendor's product died when the firewall blocked its endpoints. I did not want to inherit that single point of failure, so the AI sits behind one small interface with six interchangeable modes: the offline mock, OpenAI, Google Gemini, Azure OpenAI, Azure Document Intelligence, and an ensemble mode that runs two providers and sends any disagreement to a human. The public demo runs on one OpenAI key. Production would run the Azure providers, the same family of models but inside the tenant the firewall already trusts. Switching between any of these is an environment variable, not a rewrite.

Why Next.js. The realistic alternatives were a React app with a separate backend service, or a Python service for the model calls. Either of those means two codebases to deploy, two sets of types to keep in sync, and API keys living in a second place. Next.js gives me one TypeScript codebase where the server side makes the model calls, so keys never reach the browser, and where the list of label fields is defined once and shared by the pipeline, the screens, and the CSV export. That single definition matters in practice: adding a field cannot silently miss a layer. The production build packs into one small container that Azure can run as-is. And Next.js is boring in the way I wanted here: any reviewer knows where to find the API route and the page.

The rest of the stack, with reasons. TypeScript in strict mode, because compliance data is full of fields that may legally be absent, and strict mode forces every might-be-missing case to be handled in code instead of discovered in production. Tailwind for styling, because the styles sit directly on each element, which means the accessibility rules from the brief (contrast, large click targets, visible focus) can be checked by reading a screen's code instead of hunting through a separate stylesheet. Vitest for tests, because the whole suite finishes in about five seconds, and a five-second suite actually gets run on every save. The one extra server dependency is sharp, an image library; it does the crop-and-enlarge work for the warning check. And there is no database at all. Marcus said standalone with no personal data, and the cleanest way to protect data you do not need is to never store it.

Vercel for the demo, Azure for the real thing. I wanted a reviewer to reach the running app in one click, and Vercel deploys this repository's main branch automatically. But it is only the demo. The firewall requirement means production belongs inside TTB's Azure tenant, so the repo ships a multi-stage Dockerfile and documented paths for Azure App Service and Container Apps.

What I would do next. First the unglamorous step: deploy the existing container in-tenant with reserved model quota, behind the agency gateway that already handles login and rate limiting. Then layout-aware OCR, which would unlock the checks I deliberately skipped because they cannot be judged from extracted text alone, like minimum type sizes. Then a standing audit: sample a slice of the labels that went through as Approve for a second human look, so the false-approval rate keeps being measured in production. COLA integration goes last, because getting access to that system is paperwork before it is engineering.

That is the habit underneath the whole project: measure before choosing, and hand every uncertain case to a person.